

Differential Symbolic Execution*

Suzette Person, Matthew B. Dwyer,
Sebastian Elbaum
Dept. of Computer Science and Engineering
256 Avery Hall
University of Nebraska
Lincoln, NE 68588-0115
{sperson,dwyer,elbaum}@cse.unl.edu

Corina S. Păsăreanu
NASA Ames Research Center
Mail Stop 269-2
Moffet Field, CA 94035
pcorina@email.arc.nasa.gov

ABSTRACT

Detecting and characterizing the effects of software changes is a fundamental component of software maintenance. Version differencing information can be used to perform version merging, infer change characteristics, produce program documentation, and guide program re-validation. Existing techniques for characterizing code changes, however, are imprecise leading to unnecessary maintenance efforts.

In this paper, we introduce a novel extension and application of symbolic execution techniques that computes a precise behavioral characterization of a program change. This technique, which we call *differential symbolic execution* (DSE), exploits the fact that program versions are largely similar to reduce cost and improve the quality of analysis results. We define the foundational concepts of DSE, describe cost-effective tool support for DSE, and illustrate its potential benefit through an exploratory study that considers version histories of two Java code bases.

Categories and Subject Descriptors

D.2.5 [Software Engineering]: Testing and Debugging—*symbolic execution*; D.2.7 [Software Engineering]: Distribution, Maintenance, and Enhancement—*documentation*

General Terms

Documentation, Verification

Keywords

Program Differencing, Symbolic Execution, Software Evolution

*This material is based in part upon work supported by the National Aeronautics and Space Administration under grant number NNX08AV20A and by the National Science Foundation under awards CCF-0429149, CNS-0454203, CCF-0541263, and CNS-0720654. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the NASA or NSF.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

SIGSOFT 2008/FSE-16, November 9–15, Atlanta, Georgia, USA
Copyright 2008 ACM 978-1-59593-995-1 ...\$5.00.

1. INTRODUCTION

Detecting and characterizing the effects of software changes is a fundamental component of software maintenance. Version differencing information can be used to perform version merging, infer change characteristics, produce program documentation, and guide program re-validation. For example, during regression testing, differences can be used to focus re-testing efforts by selecting only test cases that exercise the modified sections of code. Reducing the scope of maintenance activities by exploiting version difference information, can significantly reduce maintenance costs.

Computing version differences involves comparing two programs by **systematically matching representations**. Differences can be computed on a variety of representations (e.g., flow graphs [17], abstract syntax trees [2], lines in program source files [9]) of program versions. Calculating a source file *diff* is the most common approach used in practice and while it is automated and efficient, it is very imprecise. Formatting or minor syntactic changes to a program, that do not alter its meaning, can cause a diff to indicate version differences. Subsequent maintenance activities that use source differences as their starting point to identify the effects of a change may, consequently, do large amounts of unnecessary work. We believe that recent advances in behavioral program analysis techniques, more specifically those associated with symbolic execution [14], make it possible to produce significantly more precise version differencing information.

In this paper, we present a novel technique, *differential symbolic execution* (DSE), that combines state-of-the-art symbolic execution approaches with a new over-approximating symbolic execution technique to characterize the effects of a set of program changes in terms of behavioral program differences. **DSE supports multiple forms of equivalence checking, and is capable of precisely characterizing behavioral program differences when equivalence cannot be shown.** Moreover, our formulation of DSE allows it to “skip” the symbolic execution of the parts of a program that are unchanged between versions. This has the potential to significantly reduce analysis cost and to avoid some of the fundamental limitations of symbolic execution. Finally, DSE supports a wide range of program evolution tasks including the problems of equivalence checking and regression test generation.

The contributions of our work lie primarily in the development of a methodology for performing DSE and the foundational concepts that enable DSE to leverage advances in symbolic execution to precisely characterize the changes between two program versions. Specifically, our work: (1) **develops a methodology for precisely characterizing behavioral program differences using symbolic execution**, (2) **develops techniques to compute over-approximating symbolic method summaries by identifying and automatically summarizing the behavior of common program fragments**, (3) **defines**

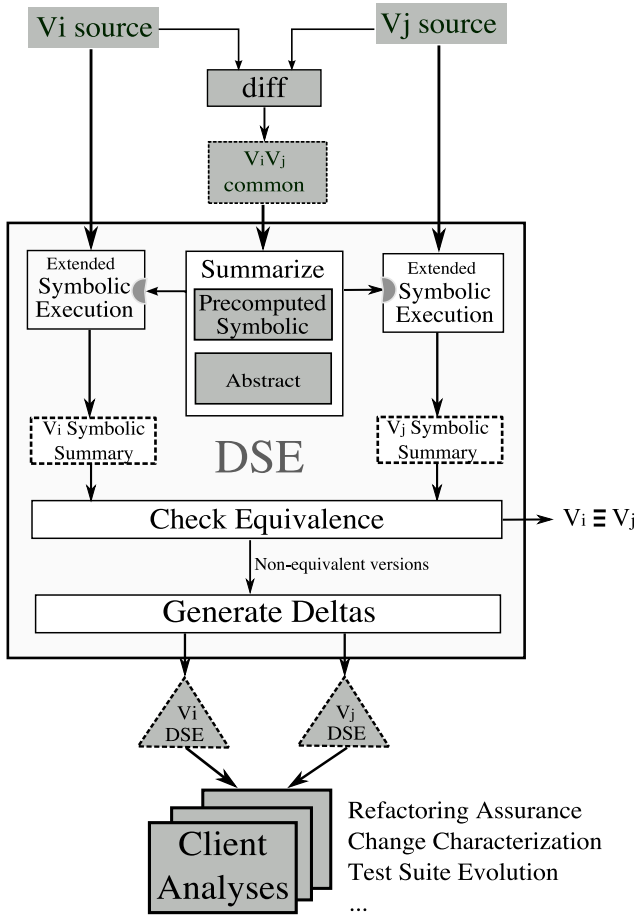


Figure 1: DSE Methodology

two behavioral equivalences between program versions and two characterizations of behavioral version differences, (4) develops techniques for post-processing symbolic execution results to compute those differences, (5) defines the conditions under which DSE analysis results completely account for program behavior and, importantly, when they do not, and (6) describes three applications of DSE results to support the automation of program evolution tasks. We have implemented support for several DSE techniques in Java PathFinder’s symbolic execution infrastructure [16] and report on the findings of an exploratory study applying DSE techniques to reason about version histories from two real Java systems.

In the next section, we provide a high level overview of differential symbolic execution, its application to a small example, and describe opportunities for integrating DSE results into existing program evolution and maintenance tasks.

2. OVERVIEW

In this section, we provide an overview of the methodology for performing differential symbolic execution and illustrate its application to a small example. We begin with a brief explanation of symbolic execution.

2.1 Symbolic Execution

Symbolic execution [14] is a non-standard approach to executing programs that uses *symbolic values* instead of actual values as

inputs, and computes output values expressed as a function of the symbolic input values. Symbolic execution of the code fragment:

```
y = x;
if (y > 0) then y++;
return y;
```

would introduce a symbolic variable X to denote the value of variable x on entry to the fragment. Symbolic execution calculates two behaviors: (1) when $X > 0$ the value $X + 1$ is returned and (2) when $!(X > 0)$ the value X is returned; note that references to variable y are replaced by its value expressed in terms of X . The behaviors are represented as the pairs $(X > 0, RETURN == X + 1)$ and $(!(X > 0), RETURN == X)$, where the first element of a pair encodes the conjunction of constraints along an execution path, i.e., the *path condition*, and the second element defines how the code modifies the program variables, e.g., $RETURN == X$ means that the original value for x is returned. This set of pairs represents a *symbolic summary* for the code fragment; the summary is *complete* since it accounts for all possible input values that influence the behavior of the fragment.

2.2 Our Approach

The DSE methodology performs a form of *differential* program analysis [15, 29] that leverages advances in symbolic execution [1, 11, 16, 27, 28] to precisely characterize the behavior of one program version relative to another. This analysis is capable of (a) demonstrating that the two versions are equivalent, or, if they are not, of (b) characterizing the behavioral differences between versions by identifying the sets of inputs that cause a different effect. We illustrate the principles and several applications of our analyses here.

Figure 1 illustrates the steps involved in performing our analysis. In its simplest form, DSE consists of 3 steps. First, it takes two versions of a method, V_i and V_j , and symbolically executes each method version to generate symbolic summaries. Second, it checks whether the summaries are equivalent. As we explain in Section 4, we define *functional* equivalence, i.e., the versions have the same black-box behavior, and *partition-effects* equivalence, i.e., the versions have corresponding sets of paths through their implementations. These notions of equivalences are useful for different applications, and each is checked by encoding summaries as a series of validity queries to an automated decision procedure, e.g., CVC3 [7]. The third step involves generation of a behavioral delta by DSE, denoted Δ , if the summaries are not equivalent. The delta precisely characterizes the input values on which the versions exhibit different behaviors.

Consider the method `logicalValue()` in Figure 2. For version 1, a symbolic execution that bounds the loop to at most 2 iterations, results in a summary consisting of 5 pairs:

```
(!(CT - T >= 100), RETURN == O)
(!(CT - T >= 100) ∧ D == null, RETURN == NRE)
(!(CT - T >= 100) ∧ !D(== null) ∧ !(0 < D.L),
old == 0 ∧ RETURN == 0)
(!(CT - T >= 100) ∧ !D(== null) ∧ 0 < D.L ∧ !(1 < D.L),
old == D[0] ∧ RETURN == D[0])
(!(CT - T >= 100) ∧ !D(== null) ∧ 0 < D.L ∧
1 < D.L ∧ !(2 < D.L),
old == D[0] + D[1] ∧ RETURN == D[0] + D[1])
```

where CT , D , $D.L$, T represent the symbolic values of `currentTime`, `data`, `data.length` and `t` on entry to the method, and NRE stands for `NullPointerException`. A similar calculation for version 2, produces a nearly identical summary, except that the first pair has $CT - T < 100$ rather than

Version 1	Version 2	Version 3
<pre> public int logicalValue(int t) { if (!(currentTime - t >= 100)) { return old; } else { int val = 0; for (int i=0; i < data.length; i++){ val = val + data[i]; } old = val; return val; } } </pre>	<pre> final int THRESHOLD = 100; public int logicalValue(int t) { int elapsed = currentTime - t; int val = 0; if (elapsed < THRESHOLD) { val = old; } else { for (int i=0; i < data.length; i++){ val = val + data[i]; } old = val; } return val; } </pre>	<pre> final int THRESHOLD = 100; public int logicalValue(int t) { int elapsed = currentTime - t; int val = 0; if (elapsed < THRESHOLD) { val = 1; } else { for (int i=0; i < data.length; i++){ val = val + data[i]; } old = val; } return val; } </pre>

Figure 2: Three Versions of `logicalValue()`

!($CT - T \geq 100$) and the remaining pairs have !($CT - T < 100$) rather than !($CT - T \geq 100$). We use automated reasoning tools to judge equivalence, but in this case, it is relatively easy to observe that the summaries are equivalent up to the precision of the symbolic execution. We note, however, that since the symbolic execution did not consider behaviors for inputs where $D.L > 2$ the summaries used to judge equivalent are *incomplete* and so too is the judgment of equivalence; we return to this point later in this section.

There are a wealth of potential client applications for DSE. Consider version 2 of `logicalValue()`, which is the result of applying three of Fowler’s code refactorings [10] to version 1: *Replace Magic Number with Symbolic Constant*, *Introduce Explaining Variable*, and *Reverse Conditional*. Refactorings are intended to preserve the behavior of a program, but as in this case, they can give rise to significant textual and syntactic differences in the program. For instance, applying the standard `diff` tool reveals that half of the source lines across versions 1 and 2 of `logicalValue()` changed; common lines include block C, the method header, the `else`, and closing `}`. On the other hand, DSE determines the two versions to be equivalent which provides assurance that refactorings have been applied correctly; experience has shown that producing correct refactorings can be difficult when performed manually or when using tool support [8].

Client analysis such as impact analysis [19] and regression test selection [20] could also leverage DSE’s more precise deltas to reduce the impact set size or the number of tests selected for execution. Moreover, the partition-effects equivalence of versions 1 and 2 of `logicalValue()` indicates that any test used on version 1 provides equivalent coverage on version 2. Consequently, the refactoring neither invalidates existing tests nor calls for new tests to achieve a desired coverage goal.

Deltas can also be used to provide a concise and precise symbolic characterization of a change. For example, in moving from version 2 to version 3, DSE produces deltas that can be processed to produce the change explanation:

On input	Version 2	Version 3
<code>currentTime - t < 100</code>	returns old	returns 1

This could be used, for example, to confirm the intention of a change with a developer on version checkin or to provide version documentation.

Symbolic execution is a powerful technique, but it does have limits. A complete symbolic execution of the `for` loop in method `logicalValue()` requires knowing how many iterations of the

loop are performed which is generally unknown. Moreover, the standard approach to symbolic execution does not leverage the fact that code versions may share significant blocks of common code. When program features that are problematic for symbolic execution lie within code that is common to multiple versions, DSE can provide significant benefits.

The DSE methodology uses the output of a *diff* tool (e.g., string, bytecode, syntax tree based) to identify blocks of code that are common to both versions. For example, applying *diff* to versions 1 and 2 of method `logicalValue()` in Figure 2 identifies the shaded regions as a common block of text between versions.

DSE exploits the commonality between versions to *summarize* the behavior of the common block. For our example, we use an *abstract* summary for the common code; a previously computed *symbolic* summary could also be used. An abstract summary models the common block C using an uninterpreted function [4] that takes values for the `data` field and the local variable, `val`, and returns a new value for fields `old` and `val`; the inputs and outputs of a common code block can be determined by static analysis, e.g., a side-effects analysis [18]. An abstract summary represents behavior as a pair of constraints; for C in `logicalValue()`, we have the pair:

$$(IP_C(D, V), old = old_C(D, V) \wedge val = val_C(D, V))$$

where D and V represent the symbolic value of `data` and `val` on entry to the block. The IP function represents the path condition within block C and the functions $xc_C(D, V)$ represent the resultant values for variable x when the block completes execution.

Uninterpreted functions are free to take on any value and, consequently, an abstract summary will over-approximate the behavior of C. Uninterpreted function applications with the same parameter values can, however, be equated. This allows the behavior of block C in version 1 to be equated to the behavior of the block in version 2. Moreover, the behaviors can be judged equivalent without ever performing a detailed symbolic execution of C which would be problematic for symbolic execution due to the `for` loop.

Abstract summaries are input to an extended form of symbolic execution which is able to combine abstract summaries with traditional symbolic execution calculations to produce a method summary for each version. For version 1, the DSE summary consists of two pairs:

$$\begin{aligned}
 &(! (CT - T \geq 100), RETURN = O) \\
 &(! (CT - T \geq 100) \wedge IP_C(D, V), \\
 &\quad old = old_C(D, V) \wedge RETURN = val_C(D, V))
 \end{aligned}$$

Calculating the DSE summary for version 2, results in a slightly different summary:

$$\begin{aligned} & (CT - T < 100, RETURN = O) \\ & (! (CT - T < 100) \wedge IP_c(D, V), \\ & \quad old = old_c(D, V) \wedge RETURN = val_c(D, V)) \end{aligned}$$

and for version 3 we have:

$$\begin{aligned} & (CT - T < 100, RETURN = 1) \\ & (! (CT - T < 100) \wedge IP_c(D, V), \\ & \quad old = old_c(D, V) \wedge RETURN = val_c(D, V)) \end{aligned}$$

These summaries were computed by over-approximating the behavior of the common code block with abstract summaries. As a consequence, they are complete with respect to the input partition of `logicalValue()`. Thus, equivalence judged based on these summaries will be sound.

It is easy to observe the equivalence between versions 1 and 2, and one can quickly identify the delta between versions 2 and 3 since the first pair differs. Using abstract summaries demands a decision procedure that supports uninterpreted functions, such as CVC3. The delta for version 3 relative to version 2 is

$$\Delta_{v3} = \{(CT - T < 100), RETURN = 1\}$$

This delta characterizes the set of inputs for which the behavior of the method is different and the resultant values computed for those inputs. Note that this delta does not include any terms that reference uninterpreted functions (abstract summaries) – those were only present in pairs that were common to both versions. In deltas with uninterpreted function terms, it is possible to further difference pairs to eliminate these terms or to refine such terms by substituting symbolic summaries on a demand-driven basis [1].

In the following sections, we define the foundational concepts of DSE beginning with summary calculations (Section 3) then notions of equivalence and delta (Section 4). We then define multiple forms of DSE and describe DSE tool support that we have built in Section 5. In Section 6, we report findings drawn from an exploratory study of version histories of two Java code bases and detail qualitative findings that suggest the potential of DSE in supporting software maintenance activities. We relate our work to approaches described in the literature in Section 7 and outline our plans for follow-on work in Section 8.

3. SUMMARIZING PROGRAM BEHAVIOR

We represent the behavior of a program (or a part of a program) using a *symbolic summary*. Conceptually, a symbolic summary captures a set of distinct cases of a program’s behavior, where each case is characterized in terms of a set of input values and the corresponding set of effects on program variables. The presence of loops, recursion, complex arithmetic expressions, and other program structures for which sophisticated theories are necessary in order to reason about program behavior may result in symbolic summaries that vary in (a) *precision* – one summary is more precise than another if it represents less infeasible behavior (i.e., the summary includes fewer infeasible paths), and (b) *completeness* – one summary is more complete than another if it represents more feasible behavior (i.e., it characterizes program behavior over a larger

portion of its input space). Our summaries account for these dimensions of variation.

In the rest of this section, we describe the underlying symbolic execution framework for generating symbolic summaries and how DSE employs established symbolic execution techniques [1, 11, 16, 27, 28] to produce symbolic summaries. We describe how these techniques can give rise to *incomplete* summaries for program structures that occur relatively frequently in practice. We then introduce the notion of an *abstract* symbolic summary that sacrifices precision in order to gain completeness. In our presentation, we focus on the summarization of individual program methods; the same concepts can be applied to whole programs or parts of methods as well.

3.1 Symbolic Program State

Symbolic execution calculates values for program variables as expressions defined over constants and symbolic variables using a given set of operators. A symbolic variable denotes an unspecified typed value; in this work we denote symbolic variables with capital letters. During symbolic execution, symbolic variables are introduced whenever an unspecified value flows into a method, e.g., through parameters. For example, T in Section 2.2 denotes the symbolic value of parameter t on entry to a call of method `logicalValue()` in Figure 2. We associate symbolic variables with memory locations that are written to, but are not local to a method, e.g., side-effected globals or the return value of a method. For example, $RETURN$ in Section 2.2 denotes the symbolic return value of method `logicalValue()` in Figure 2. We refer to the set of locations written to (or read by) a method, m , called on arguments $\vec{a}$, as $Write(m, \vec{a})$; $Write(m)$ denotes the locations that may be written on any call.

The state of a symbolic execution is defined as $(l, pc, \vec{v})$ where l , the current location, records the next statement to be executed, pc , the path condition, is the conjunction of branch conditions encoded as constraints along the current execution path, and $\vec{v}$, the symbolic value store, is a vector of symbolic expressions representing the current values of memory locations, where $v[j]$ is the value at location j . For simplicity we assume that memory locations can be indexed and we define a symbolic value store update, $\vec{v}(d, s)$, to be $\vec{v}'$ where $(\forall d' \neq d : v'[d'] = v[d']) \wedge (v'[d] = s)$; similarly $\vec{v}(e)$ updates the store to be consistent with the effects of constraint e .

Computation statements, $v[d] = v[j] \text{ op } v[k]$, when executed symbolically in state $(l, pc, \vec{v})$ produce a new state

$$(l + 1, pc, \vec{v}(d, \text{op}(v[j], v[k])))$$

that reflects the update of the value store with a new symbolic value for d that applies op to its operand values. Branching statements, *if* $v[j] \text{ op } v[k]$ *goto* d , when executed symbolically in state $(l, pc, \vec{v})$ branch the symbolic execution to *two* new states $(d, pc \wedge v[j] \text{ op } v[k], \vec{v})$ and $(l + 1, pc \wedge \neg(v[j] \text{ op } v[k]), \vec{v})$ corresponding to the “true” and “false” evaluation of the branch condition, respectively. An automated decision procedure is used to check the satisfiability of the updated path conditions and, when a path condition is found to be unsatisfiable, symbolic execution along that path halts. Decision procedures for a range of theories that underly constraint languages, such as, linear arithmetic, arrays, and equality over uninterpreted functions, are readily available, e.g., CVC3 [7]. While these procedures can form an effective basis for symbolic execution of non-trivial programs, there are many data types and operations that require theories for which cost-effective decision procedures are not currently available e.g., non-linear arithmetic (this is one of the reasons we use abstract summaries as described later in this section).

3.2 Symbolic Summaries

Scalability is a key concern in symbolic execution, especially in an inter-procedural setting; we address it by adapting recent work on compositional symbolic execution [11, 1]. Intuitively, a method summary defines “what” a method computes by pairing descriptions of input values with the changes to the program state effected by executing the method on those values.

The basic building block of a symbolic summary is the *constraint*. Each constraint defines either the set of concrete values for method inputs, or the set of effects, e.g., return values caused by method execution. Constraints consist of conjunctions of boolean expressions defined in the constraint language, and are combined to form a *partition-effects pair*.

DEFINITION 3.1 (PARTITION-EFFECTS PAIR). A *partition-effects pair*, (i, e) , consists of: an input constraint, i , which is a conjunction of relational expressions defined over constants and symbolic variables, and an effects constraint, e , which is a conjunction of expressions that equate written locations to expressions defined over constants and symbolic variables.

We note that such a pair effectively defines a pair of pre- and post-conditions; summaries in [11, 1] are defined as $i \wedge e$, but we separate them to make it possible to compute with i and e independently as discussed in Section 4.

Intuitively, the input constraint defines a set of values for method inputs. The effect constraint defines the bindings of values, represented symbolically, to effected locations. Sets of these partition-effect pairs serve to summarize the behavior of a method.

DEFINITION 3.2 (SYMBOLIC SUMMARY). A *symbolic summary*, for a method m , is a set partition-effects pairs $m_{sum} = \{(i_1, e_1), (i_2, e_2), \dots, (i_k, e_k)\}$ where the input constraints are disjoint, i.e., $\forall 1 \leq j \leq k \forall 1 \leq j' \leq k \wedge j \neq j' : i_j \wedge i_{j'}$ is unsatisfiable.

A symbolic summary captures the partitions of the input space of a method and associates with each partition, a symbolic encoding of the precise effects of executing the method on an input in that partition. Summaries are updated during symbolic execution whenever an intra-method path reaches a return statement. At that point the partition-effects pair, $(pc, \bigwedge l \in Write(m) : l = v[l])$, is added to the summary to capture the effects along the path.

Executing a method call, $x = m(\vec{a})$, symbolically in state $(l, pc, \vec{v})$ with summary m_{sum} involves considering each partition-effects pair in m_{sum} in turn and judging whether it represents the behavior of m that is invoked at the call site and, if so, applying its effects on the symbolic program state. Let $\vec{a}$ be the symbolic expressions associated with actual parameters of the call and $\vec{f}$ be the symbolic values associated with the formal parameters of m that are used in m_{sum} . Let $ret \in Write(m)$ be the symbolic variable holding m 's return value. For each pair $(i, e) \in m_{sum}$, we form $pc' = pc \wedge (\bigwedge j \in |\vec{a}| : a[j] == f[j]) \wedge i$ which conjoins the current path condition with a set of equality constraints that equate symbolic variables for the formal parameters with the symbolic expressions for the actuals, and with the input partition, i . If pc' is unsatisfiable then the behavior of the pair is inconsistent with the calling context and we ignore it. If pc' is satisfiable we continue symbolic execution to produce the state:

$$(l + 1, pc', \vec{v}(x == ret \wedge e))$$

This state updates the store to be consistent with the effects constraint, e , and an additional equality constraint which forces the return value, ret , to update the location for x . Multiple pairs in a

summary may be consistent with the current pc and each pair will produce an independent continuation of the symbolic execution.

3.3 Incomplete Summaries

Unfortunately, it is not always possible to calculate a complete summary using symbolic execution. There are two main obstacles: (1) cost-effective automated decision procedures may be unavailable for theories needed to model certain program structures (e.g., non-linear arithmetic) and (2) loops and recursive method invocations, in general, are impossible to bound using input constraints. The inability to reason about these structures results in summaries that are incomplete with respect to reasoning about a method's input space; the loop-iteration bounded summaries in Section 2 illustrate this problem.

The completeness of a summary is based on the degree to which the input partitions in the summary cover the method's input space.

DEFINITION 3.3 (COMPLETE AND INCOMPLETE SUMMARY). A symbolic summary, for a method m , m_{sum} is complete if

$$\bigvee (i, e) \in m_{sum} : i$$

is valid; otherwise the summary is incomplete.

There are two strategies one can adopt to compensate for incomplete summaries: (1) explicitly define the portion of the method input space that is covered by the incomplete summary and then restrict reasoning results to that space (which we explore in Section 4.3), and (2) develop techniques to produce complete summaries (which we explore in Section 3.4).

To support the first approach, we define the portion of a method's input space that is accounted for by a summary.

DEFINITION 3.4 (INPUT PARTITION COVERAGE). Given a method summary m_{sum} the input partition coverage is

$$\iota(m_{sum}) = \bigvee (i, e) \in m_{sum} : i$$

One can always increase the input partition coverage of a symbolic summary by tuning the symbolic execution. In applications to test generation, researchers have explored this idea to increase the path-coverage for a generated test suite by, for example, using larger bounds on the unrolling of loops in symbolic execution [27, 28].

3.4 Abstract Summaries

Symbolic summaries are as precise as the underlying theories implemented by the decision procedures used. It is possible to define less precise symbolic summaries, but researchers applying symbolic execution to testing have not found value in this approach since they rely on symbolic execution for exploring only feasible paths, forming an under-approximation of program behavior. In this section, we develop an over-approximating symbolic summary, that may include infeasible paths, but is useful in differential symbolic execution.

An *abstract symbolic summary* captures the fact that a method (or block of code) may partition its input space, and within each partition it produces effects on program locations; in other words, it abstracts the number of partitions and the specific nature of each partition and its associated effects.

DEFINITION 3.5 (ABSTRACT SUMMARY). An abstract summary, for a method $m(\vec{f})$ with parameters $\vec{f}$ is a singleton set of partition-effects pairs

$$m_{abs} = \{(i(\vec{f}), \bigwedge l \in Write(m) : l == e_l(\vec{f}))\}$$

where $i : \vec{f} \rightarrow \{true, false\}$, and $e_l : \vec{f} \rightarrow \text{typeof}(l)$ are uninterpreted functions defined over vectors of parameter values.

This summary generates an uninterpreted function that represents each effect of executing m on a given input. When symbolically executing two different calls to a method with equivalent symbolic parameter expressions, the effects of those calls can be judged equivalent using decision procedures that apply functional consistency in the theory of uninterpreted functions [4].

For example, the invocations of block C in Figure 2 in version 1 and version 2 have the same actual parameters, SE and V , so their effects, $\text{old}_C(SE, V)$ and $\text{val}_C(SE, V)$, are equivalent.

Executing a method call to, $x = m(\vec{a})$, symbolically in state $(l, pc, \vec{v})$ with m_{abs} involves parameterizing the uninterpreted functions and then generating the new state:

$$(l + 1, \\ pc \wedge i(\vec{a}) \wedge (\bigwedge l \in \text{Write}(m) : v_l = e_l(\vec{f})), \\ \vec{v}[x == v_{ret} \wedge (\bigwedge l \in \text{Write}(m) - \{ret\} : l = v_l)])$$

Here the binding of formals to actuals is achieved by parameterizing the uninterpreted functions. Note that we introduce fresh symbolic variables v_l and equate them in the new path condition to the computational effects of the abstracted call. Those values are then equated to the effected locations to update the symbolic value store; the return location ret is singled out to update the assigned location x . For effects produced by abstract summaries, this realizes a kind of “single assignment” representation along paths in the symbolic execution. This representation creates larger, but much simpler constraints to solve, and has other advantages as discussed in Section 6.4.

Since symbolic execution halts whenever an unsatisfiable path condition is encountered, the only allowable interpretation of $i(\vec{a})$ is that it is valid. Consequently, we view abstract summaries as (trivially) complete. As discussed in Section 5, when expanding an abstract summary with an incomplete symbolic summary completeness may be lost.

Clearly abstract summaries may be less precise than a symbolic summary. For example, deciding conditionals based on an effect encoded in m_{abs} may cause subsequent infeasible branching in the symbolic execution. On the other hand, if precision for a particular call is not required in a differential analysis, the fact that abstract summaries are complete makes them an ideal way to represent a method or code block when it cannot be executed symbolically.

4. EQUIVALENCES AND DELTAS

In this section, we define two notions of equivalence between methods (or code blocks) based on summaries. *Functional* equivalence relates methods that have the same externally observable behavior, whereas the finer-grained notion of *partition-effects* equivalence takes into account the way that a method’s implementation partitions its input space. Associated with each type of equivalence is a corresponding notion of *delta* which captures the behavioral differences between methods symbolically.

For the most part, our definitions are insensitive to the use of symbolic or abstract summaries. In practice, as we discuss in Section 5, we use abstract summaries in DSE to capture the behavior of the portions of a program that are unchanged between versions, and symbolic summaries for changed portions. Given this, summaries will, in general, have constraints built by composing both symbolic and abstract summaries.

4.1 Functional Equivalence and Delta

Like black-box approaches to testing, **functional equivalence ignores the internal details of a method, e.g., how it is structured, and focuses on “what” effects it computes for a given input.** We say that two versions of a method, m and m' , are **functionally equivalent** if the collective partition-effects pairs in m_{sum} is logically equivalent to the collective partition-effects pairs in m'_{sum} .

Godofroid [11] developed the notion of logical method summary. We restate it here since it forms the basis for judging functional equivalence.

DEFINITION 4.1 (LOGICAL METHOD SUMMARY [11]). Given a method, m , and its symbolic summary, m_{sum} the logical method summary for m is

$$\langle m \rangle = \bigvee (i, e) \in m_{sum} : i \wedge e$$

Functional equivalence relates two logical method summaries.

DEFINITION 4.2 (FUNCTIONAL EQUIVALENCE). Given two methods m and m' , m is functionally equivalent to m' if and only if $\langle m \rangle \Leftrightarrow \langle m' \rangle$

Depending on the nature of the constraints in method summaries, different forms of automated decision procedures can be applied to check for this equivalence.

If functional equivalence does not hold, then m and m' produce different effects for some input(s). We capture these differences as a functional delta between methods.

DEFINITION 4.3 (FUNCTIONAL DELTA). Given two methods, m and m' , the functional delta is

$$\Delta_{\langle m, m' \rangle} = \langle m \rangle \wedge \neg \langle m' \rangle$$

Intuitively, a delta describes functionality that is present in one version of the method and is not present in the other version of the method. Thus, we can define a delta from m ’s point of view, $\Delta_{\langle m, m' \rangle}$, which captures behavior in m that is not in m' , or from m' ’s point of view $\Delta_{\langle m', m \rangle}$. For convenience, when the pair of methods m and m' is clear from the context, we use $\Delta_{\langle m \rangle} = \Delta_{\langle m, m' \rangle}$ and $\Delta_{\langle m' \rangle} = \Delta_{\langle m', m \rangle}$.

4.2 Partition-effects Equivalence and Delta

If one wishes to reason about the path-equivalence of two versions of a method to, for example, infer coverage properties of an existing test suite on the new version, then functional equivalence is inadequate. Partition-effects equivalence, on the other hand, considers both “what” a method does and “how” it partitions the input space, therefore partition-effects equivalence considers the equivalence of each method’s input partition independently.

DEFINITION 4.4 (PARTITION-EFFECTS EQUIVALENCE). Two methods, m and m' , with method summaries m_{sum} and m'_{sum} are partition-effects equivalent if and only if

$$\begin{aligned} \forall (i, e) \in m_{sum} \exists (i', e') \in m'_{sum} : (i \wedge e) \Leftrightarrow (i' \wedge e') \\ \forall (i', e') \in m'_{sum} \exists (i, e) \in m_{sum} : (i' \wedge e') \Leftrightarrow (i \wedge e) \end{aligned}$$

This equivalence requires an isomorphism between method summaries where each corresponding pair of summary elements is logically equivalent.

Partition-effects equivalence is finer-grained than functional equivalence. For example, the refactoring example above illustrates that two functionally equivalent methods are partition-effects *inequivalent*. Two partition-effects equivalent methods are guaranteed to be functionally equivalent.

If partition-effects equivalence does not hold for a pair of methods, m and m' , then either they are not functionally equivalent or, the methods partition the input space differently, but produce the same effects in the corresponding partitions. We capture the differences at the input partition level with partition-effects deltas.

DEFINITION 4.5 (PARTITION-EFFECTS DELTA). *Given two versions of a method, m and m' , the partition-effects delta is:*

$$\Delta_{m,m'} = \{(i, e) \mid (i, e) \in m_{sum} \wedge \neg \exists (i', e') \in m'_{sum} : (i \wedge e) \Leftrightarrow (i' \wedge e')\}$$

This delta produces a set of partition-effects pairs that reflect input partitions and their associated effects, present in one version of the method that are not present in the other version of the method. As with functional deltas, we define partition-effects delta from both m and m' 's perspective and when m and m' are clear from the context, we use $\Delta_m = \Delta_{m,m'}$ and $\Delta_{m'} = \Delta_{m',m}$. For versions 2 and 3 of `logicalValue()` from Section 2.2:

$$\begin{aligned} \Delta_{v3} &= \Delta_{v3,v2} = \{(CT - T < 100, RETURN == 1)\} \\ \Delta_{v2} &= \Delta_{v2,v3} = \{(CT - T < 100, RETURN == 0)\} \end{aligned}$$

In this work, we introduce only two types of equivalences and deltas because they are appropriate for the applications presented in Section 6. We note, however, that variations on equivalences and deltas are possible. For example, one could extend the definition of partition-effects equivalence to consider many-to-many correspondences between partition-effects pairs in method summaries rather than just one-to-one correspondences.

4.3 Handling Incomplete Summaries

Partition-effects equivalence can be judged relative to the input partition that is covered by both method summaries.

DEFINITION 4.6 (RELATIVE PARTITION-EFFECTS EQUIV.). *Two methods, m and m' , with method summaries m_{sum} and m'_{sum} are relative partition-effects equivalent if and only if*

$$\begin{aligned} \forall (i, e) \in m_{sum} \exists (i', e') \in m'_{sum} : \\ (i \Rightarrow (\iota(m_{sum}) \wedge \iota(m'_{sum}))) \Rightarrow ((i \wedge e) \Leftrightarrow (i' \wedge e')) \\ \forall (i', e') \in m'_{sum} \exists (i, e) \in m_{sum} : \\ (i' \Rightarrow (\iota(m_{sum}) \wedge \iota(m'_{sum}))) \Rightarrow ((i' \wedge e') \Leftrightarrow (i \wedge e)) \end{aligned}$$

This definition relaxes the requirement that a partition-effects pair in one summary has a corresponding pair in the other summary, when that pair's input constraint defines a set of input values that does not lie completely within the intersection of the method summaries covered input partitions. The delta can also be defined relative to the covered input partition.

DEFINITION 4.7 (RELATIVE PARTITION-EFFECTS DELTA). *Given two versions of a method, m and m' , the relative partition-effects delta is:*

$$\{(i, e) \mid (i, e) \in \Delta_{m,m'} \wedge i \Rightarrow (\iota(m_{sum}) \wedge \iota(m'_{sum}))\}$$

For complete summaries, these definitions simplify to those provided in the previous subsection.

While it is often possible to increase the input partition coverage of symbolic summaries to yield more information, one cannot always produce complete summaries. In such cases, it is clear that the use of incomplete summaries limits the ability to judge equivalence or to precisely characterize the differences between two method versions. DSE using incomplete summaries, unlike many other forms of program analysis, is able to precisely characterize the conditions under which it does not provide useful analysis results.

DEFINITION 4.8 (UNCOVERED INPUT PARTITION).

Given two versions of a method, m and m' , with method summaries m_{sum} and m'_{sum} , the uncovered input partition is $\neg(\iota(m_{sum}) \wedge \iota(m'_{sum}))$.

The uncovered input partition can be used to target subsequent analysis to program behavior that lies within the partition.

5. DIFFERENTIAL SYMBOLIC EXECUTION

The previous sections introduce a range of approaches for the generation of symbolic summaries, for equating summaries of method versions, and for characterizing the deltas between versions. This flexibility makes it possible to instantiate multiple variations of differential symbolic execution.

5.1 DSE Variations

DSE is the coordinated application of symbolic execution to versions of a program method to produce summaries of method behavior that can be equated and differenced. We highlight three of the many possible variations of DSE in this section.

5.1.1 Intra-procedural DSE

This approach performs symbolic execution along intra-procedural paths for the method versions. It does not use method summaries of any form and it is the most direct application of existing symbolic execution technologies [16]. For methods with complex non-looping branching logic and few calls, or where most of the methods (and called methods) undergo significant change, this can be an effective strategy. In general, this approach may lead to incomplete summaries and, consequently, only relative notions of equivalence and difference can be calculated.

5.1.2 Symbolic compositional DSE

This approach performs symbolic execution for the method versions using a compositional approach, e.g. [11]. For unchanged called methods, summaries from previous DSE analyses can be reused. This approach offers the potential to significantly reduce the cost of DSE, through summary reuse, relative to intra-procedural DSE. Like intra-procedural DSE, this approach will, in general, lead to incomplete summaries.

5.1.3 Abstract compositional DSE

This approach replaces common code between method versions with their abstract summaries. It then performs symbolic execution of the changed methods incorporating the abstract summaries. For methods with few changes, and especially where complex looping and calculations are performed in common code, this approach has the potential to reduce the cost of DSE and to allow a more complete summarization.

Although this approach may produce deltas that include uninterpreted functions, there are several approaches to deal with this. An uninterpreted function can be expanded by replacing it with partition-effects pairs whose input partitions match the constraints on function parameters. Pairs can be taken from pre-computed summaries or they can be calculated based on the input partition of the delta, using “on-demand” computations, e.g. [1]. Differencing corresponding pairs in a delta, e.g., to explain the nature of a change to a developer, may factor out uninterpreted functions. One approach is to remove common sub-formula between the pairs which eliminates any common uninterpreted function terms.

5.2 Tool Support

We have adapted the Java PathFinder (JPF) analysis toolset's recently developed symbolic execution framework [16] to implement support for DSE. Java PathFinder is an open source tool that acts as a customized Virtual Machine, operating on Java class files. Symbolic execution in JPF v4.1 currently supports mixed symbolic and concrete execution of integer and real data types. Symbolic execution is performed based on user specification of methods, parameters and globals to be treated symbolically. JPF traverses interprocedural paths during symbolic execution, accumulating symbolic constraints which are output for method summaries by a customized *listener* in the symbolic execution framework.

The JPF symbolic execution framework is implemented as an extension of JPF (the `symbc` JPF extension). It includes a customized bytecode instruction factory that replaces or extends the standard concrete execution semantics with symbolic execution. Symbolic information is stored in *attributes* associated with program data. JPF uses the Choco constraint programming system [5] for decision procedure support and constraint solving. Because state matching is, in general, undecidable when states represent path conditions on un-bounded data, JPF does not perform state matching on symbolic values. Loops and recursion can be bound by placing a depth limit on JPF's search depth or by limiting the number of constraints encoded for any given path; JPF indicates when one of these bounds has been reached during symbolic execution.

To perform equivalence checking and generate deltas we have developed a prototype tool that reads method summaries compiled by the customized JPF symbolic execution listener and generates inputs to CVC3. The CVC3 inputs encode completeness, equivalence and the individual checks required to calculate partition-effects deltas. CVC3 supports validity and satisfiability queries over logical formulae expressed in terms of a rich set of base types, that includes uninterpreted functions, as well as user defined types. The conversion of DSE summaries to CVC3 input is essentially a transmutation of syntax.

For partition-effects equivalence and delta calculations, we expand the pairs of partition-effects pairs from the two summaries being related into separate CVC3 queries and then combine the query results. Since summaries are always finite sets, calculating validity of the quantified formula in definitions 4.4 and 4.5 is sound.

Combining JPF's existing symbolic execution support with our prototype DSE tools realizes intra-procedural DSE. We have also developed partial support for abstract compositional DSE, however, it does not support some features of the Java language, e.g., exceptional returns from abstractly summarized methods. We also include only limited support for arrays and data structures. In the future, we plan to explore ways to handle exceptional behaviors in abstract summaries, and to incorporate support for input arrays and data structures, using the lazy initialization approach from [27].

6. APPLICATIONS OF DSE

In this section we explore the applicability of DSE to code changes made during maintenance and evolution of two existing code bases. We based our study on code examples from Siena and Apache JMeter, obtained from the Software-artifact Infrastructure Repository (SIR) [26]. Siena (Scalable Internet Event Notification Architecture) is a research project aimed at designing and constructing a generic scalable event-notification service [25]. Siena consists of approximately 6KLOC and 26 classes. Our evaluation on Siena was performed on the differences between the first five versions available from SIR. Apache JMeter is a Java desktop application that can be used to perform load testing of functional

```
Version 1
public String getName() {
1) Object userObj = getUserObject();
2) return ((JMeterGUIComponent)userObj).getName();
}

Version 2
public String getName() {
1) return ((JMeterGUIComponent)getUserObject()).getName();
}
```

Figure 3: Versions 1 and 2 of `getName()`

behavior and to measure performance [13]. JMeter consists of approximately 43KLOC and 389 classes. We used the first three versions of JMeter available from SIR.

6.1 Applicability and Completeness of DSE

We provide a characterization of the changed methods in JMeter and Siena to assess the applicability of DSE and the potential effectiveness of abstract summaries in providing more complete differential analysis in the context of real-world applications.

JMeter had 352 methods changed between versions 1 and 2. Of these changes, 80 correspond to new methods, 8 are deleted methods, 6 are methods with signature changes, and 163 are non-functional changes such as pretty printing or changes related to comments. Inspection of the remaining 95 methods containing functional changes revealed that 68 can be analyzed with DSE to either determine equivalence or describe version differences. Our assessment was conservative in that we counted only methods for which full support is currently available in DSE. For example, methods with exception handling code are not counted even though those features could be handled in future versions of our tools.

Of the 68 methods analyzed by DSE, 2 contain a loop in the common code between versions, 4 contain calls to methods which are unchanged and contain a loop, and 11 call library methods which may contain significant complexity. Applying standard symbolic execution to such methods, would generate incomplete summaries. With abstract summaries, however, those 17 methods can be completely analyzed.

In the first five versions of Siena, we saw fewer modifications but a similar trend. There were 28 changed methods, 11 were new methods, 10 were formatting changes, and 7 involved actual code changes. Of those 7 changed methods, 3 can be summarized completely with standard symbolic execution, and another 2 can be summarized completely when abstract summaries are used.

Although our assessment regarding the applicability of DSE is limited to a small number of artifacts, the results are encouraging and illustrate the potential of DSE and the use of abstract summaries to exploit the differential nature of multi-version analysis to scale to real code.

We now illustrate the application of DSE to examples selected from JMeter and Siena. Each example highlights a feature of DSE and illustrates how DSE results can be used to support selected software evolution and maintenance tasks. While we do not focus on the performance of DSE here, we note that each of the analyses discussed in this paper completed in a few seconds.

6.2 Refactoring Assurance

According to Fowler, "Refactoring is a disciplined technique for restructuring an existing body of code, altering its internal structure without changing its external behavior" [10]. From a DSE per-

spective, refactored code should be functionally equivalent to the original code. If it is not, then at least one behavior has been added to or removed from the refactored version as indicated by the DSE deltas.

Using a changed method from JMeter, shown in Figure 3, we illustrate how a refactoring change can be validated using DSE. The refactoring *Replace Assignment with Initialization* [10] was applied to the method `getName()` between JMeter versions 1 and 2. This refactoring omits the assignment of the object to a temporary variable (line 1 of version 1) and combines the call chain into a single source line in version 2.

While the source-code change looks simple, it results in a non-trivial change from 7 bytecodes, for version 1, to 4 bytecodes, for version 2. The code that is common to `getName()` resides in the bodies of unchanged called methods: `getUserObject()` and `JMeterGUIComponent.getName()`; we use the qualified name to distinguish it from the method under analysis.

To perform DSE of `getName()`, we first compute abstract summaries for the called methods. The abstract summary for `getUserObject()` consists of two uninterpreted functions:

$$\begin{aligned} & \text{Boolean } IP_{\text{getUserObject}}(JMeterTreeNode\ n) \\ & \text{Object } RET_{\text{getUserObject}}(JMeterTreeNode\ n) \end{aligned}$$

which represent the input partition and return value, respectively. The functions are parameterized by an instance of `JMeterTreeNode` to model the potential references to fields of `this`. Two uninterpreted functions also comprise the abstract summary for

`JMeterGUIComponent.getName()`. The IP function is similar and the function

$$\text{String } RET_{\text{getName}}(JMeterGUIComponent\ c)$$

represents the return value. At the end of abstract differential symbolic execution, the following, identical, DSE method summary is produced for each version of `getName()`:

$$\{IP_{\text{getUserObject}}(T) \wedge IP_{\text{getName}}(RET_{\text{getUserObject}}(T)), \\ RETURN == RET_{\text{getName}}(RET_{\text{getUserObject}}(T))\}$$

The equivalence checker (trivially) determines the two version of `getName()` to be functionally and partition-effects equivalent. Because the two versions are functionally equivalent, the developer is assured the refactoring change does not affect the external functionality of `getName()`. The developer can also infer from partition-effects equivalence, that the code coverage achieved by an existing test suite that exercises `getName()` is preserved by this refactoring change.

This simple example illustrates one refactoring change applied to JMeter; there were two instances of this type of refactoring between version 1 and version 2 of JMeter. An additional 163 of the 352 changed methods contain formatting changes that DSE would judge equivalent as well, but a text-diff would not.

6.3 Test Suite Evolution

Test suites must evolve with programs and consequently, they need to be maintained. As programs change, existing tests may become invalid and need to be replaced, new tests are added, redundant tests may be removed, and developers may invest in developing a richer test suite, for example, by adding tests to meet a stronger adequacy criterion.

```

Version 3
public static boolean match(byte[] x, byte[] y) {
1) if (x.length != y.length) return false;
2) for(int i = 0; i < x.length; ++i)
3)   if (x[i] != y[i]) return false;
4) return true;
}

Version 4
public static boolean match(byte[] x, byte[] y) {
1) if (x == null && y == null) return true;
2) if (x == null || y == null || x.length != y.length) return false;
3) for(int i = 0; i < x.length; ++i)
4)   if (x[i] != y[i]) return false;
5) return true;
}

```

Figure 4: Versions 3 and 4 of `match()`

Input Partition	Effect
$X == \text{null}$	$RETURN == \text{EXCEPTION}$
$Y == \text{null}$	$RETURN == \text{EXCEPTION}$
$!(X == \text{null}) \wedge !(Y == \text{null}) \wedge (X.l \neq Y.l)$	$RETURN == \text{FALSE}$
$!(X == \text{null}) \wedge !(Y == \text{null}) \wedge$ $!(X.l \neq Y.l) \wedge IP_{B1}(T, X, Y)$	$RETURN == RET_{B1}(T, X, Y)$

Table 1: Method Summary for `match()` version 3

We show, using the example in Figure 4 from Siena, how DSE deltas can be used to support test suite evolution. This example demonstrates DSE’s use of abstract summaries in reasoning about the equivalence of code containing a loop. It also highlights the treatment of exceptions in symbolic execution. Version 3 of `match()` has implicit exceptional behaviors (i.e. the dereferences of `x` or `y` in line 1 may cause a Java `NullPointerException` to be thrown) which create additional feasible paths. Treatment of these cases is handled explicitly in version 4 by the introduction of nullness tests. The semantics of exception throws in Java is handled by interpreting, for example, `getField` bytecodes as branching statements with an implicit test for nullness of the object being dereferenced. This is easily supported in JPF by customizing the symbolic execution of selected bytecodes.

Based on the results of a text-based diff of `match()`, lines 2-4 in version 3 and lines 3-5 in version 4 are identified as common. A side-effects analysis of this common code block reveals that `x` and `y` are read and the return value is written by the block. Consequently, we introduce an abstract summary using the following uninterpreted functions:

$$\begin{aligned} & \text{Boolean } IP_{B1}(SENP\ s, \text{byte}[]\ X, \text{byte}[]\ Y) \\ & \text{Boolean } RET_{B1}(SENP\ s, \text{byte}[]\ X, \text{byte}[]\ Y) \end{aligned}$$

At the end of differential symbolic execution, the DSE method summaries shown in Tables 1 and 2 are checked for functional and partition-effect equivalence. The methods are not partition-effects equivalent because the number of partition-effects pairs is not the same in both summaries. Our tools also calculate that the summaries are not functionally equivalent. The partition-effects deltas for `match()` are:

Input Partition	Effect
$X == \text{null} \wedge Y == \text{null}$	$\text{RETURN} == \text{TRUE}$
$X == \text{null} \wedge !(Y == \text{null})$	$\text{RETURN} == \text{FALSE}$
$!(X == \text{null}) \wedge Y == \text{null}$	$\text{RETURN} == \text{FALSE}$
$!(X == \text{null}) \wedge !(Y == \text{null}) \wedge (X.l != Y.l)$	$\text{RETURN} == \text{FALSE}$
$!(X == \text{null}) \wedge !(Y == \text{null}) \wedge !(X.l != Y.l) \wedge IP_{B1}(T, X, Y)$	$\text{RETURN} == RET_{B1}(T, X, Y)$

Table 2: Method Summary for `match()` version 4

Version 0

```

public void clear() {
1)  catClass.debug("Start : clear1");
2)  int totalChild = root.getChildCount();
3)  if (catClass.isDebugEnabled()) {
4)      catClass.debug("clear1 : total child - " + totalChild);
5)  }
6)  for (int i=0; i<totalChild; i++){ ...}
7)  resultPanel.removeAll();
8)  resultPanel.repaint();
9)  childIndex=0;
10) catClass.debug("End : clear1");
}

```

Version 1

```

public void clear() {
1)  log.debug("Start : clear1");
2)  int totalChild = root.getChildCount();
3)  if (log.isDebugEnabled()) {
4)      log.debug("clear1 : total child - " + totalChild);
5)  }
6)  for (int i=0; i<totalChild; i++){ ...}
7)  resultPanel.removeAll();
8)  resultPanel.repaint();
9)  childIndex=0;
10) log.debug("End : clear1");
}

```

Figure 5: Versions 0 and 1 of `clear()`

$$\begin{aligned}
\Delta_{v3} &= \{(X == \text{null}, \text{RETURN} == \text{EXCEPTION}), \\
&\quad (Y == \text{null}, \text{RETURN} == \text{EXCEPTION})\} \\
\Delta_{v4} &= \{(X == \text{null} \wedge Y == \text{null}, \text{RETURN} == \text{TRUE}), \\
&\quad (X == \text{null} \wedge !(Y == \text{null}), \text{RETURN} == \text{FALSE}), \\
&\quad (!(X == \text{null}) \wedge Y == \text{null}, \text{RETURN} == \text{FALSE})\}
\end{aligned}$$

Note how despite the use of an abstract summary for the common code block, these deltas provide a complete symbolic description of the differences between the two versions. We can exploit those descriptions for test evolution. A delta can be interpreted as a predicate over method inputs and outputs. Given a test suite, T , for `match()` we distinguish the sets: $T_{old} = \{t | t \in T \wedge \Delta_{v3}(t)\}$, $T_{common} = T - T_{old}$, and $T_{new} = \{t | \Delta_{v4}(t)\}$. T_{old} are the tests that potentially change for version 4, T_{common} are the tests that would produce the same results on both versions, and T_{new} describe the space of tests which capture the new behavior of version 4. Common tests do not need to be rerun on version 4, and old tests may be dropped or modified to reflect the expected results of version 4. New tests can be generated by solving the constraints in each pair of Δ_{v4} to exercise the new behavior of version 4.

6.4 Change Characterization

While refactorings are intended to preserve functional equivalence, many changes are explicitly intended to modify program behavior to fix a bug or add new functionality. For example, the deltas for `match()` can be processed to produce a characterization of the behavioral differences between version 3 and 4

On input	Version 3	Version 4
$x == \text{null} \ \&\& \ y == \text{null}$	throws NRE	returns true
$x == \text{null} \ \&\& \ y != \text{null}$	throws NRE	returns false
$x != \text{null} \ \&\& \ y == \text{null}$	throws NRE	returns false

This characterization reveals the somewhat subtle difference in treatment of null references in version 4.

Figure 5 illustrates another way in which DSE deltas can be used to characterize changes. The changes consist of replacing instances of the class `catClass` with the class `log` in `JMeter`'s `clear()` from version 0 to 1. A text-based diff of these versions reveals two common blocks: line 2 (block $B1$) and lines 6-9 (block $B2$) in each version, both of which can be summarized abstractly. In addition, we note that the body of the called methods, `debug()` and `isDebugEnabled()`, have not changed and, thus, can also be summarized abstractly. The resulting summaries for the versions of `clear()` are comprised of 2 moderately complex partition-effects pairs. The (slightly simplified) input partition for the pair that avoids line 4 in version 0 is:

$$\begin{aligned}
IP_{debug}(CC, S1) \wedge CC1 == catClass_{debug}(CC, S1) \wedge \\
IP_{B1}(R) \wedge TC1 == totalChild_{B1}(R) \wedge \\
IP_{isDebugEnabled}(CC1) \wedge \\
RET_{isDebugEnabled}(CC1) == FALSE \wedge \\
IP_{B2}(RP, TC) \wedge IP_{debug}(CC1)
\end{aligned}$$

and the effects constraint is:

$$\begin{aligned}
RESULTPANEL == resultPanel_{B2}(RP, TC) \wedge \\
CHILDINDEX == childIndex_{B2}(RP, TC) \wedge \\
CATCLASS == catClass_{debug}(CC1, S2)
\end{aligned}$$

where CC , R , and RP are the symbolic variables representing the initial values of `catClass`, `root`, and `ResultPanel`, respectively. $S1$ and $S2$ are short-hand names for the first and second string constants. $CC1$ and $TC1$ are symbolic variables that are introduced to hold values produced by abstract summaries.

This style of summary, which includes equality constraints on the path condition, realizes a single-assignment structure for each program path, i.e., each assignment yields a fresh variable. We adapted our approach to produce these summaries because they provide finer grain differencing of summaries and are easily processed by decision procedures that provide general support for uninterpreted functions and equality logic.

Δ_{v1} indicates that both pairs are different. This is expected since the change from `catClass` to `log` is not known to preserve behavior. Finer-grain differencing of the pairs in Δ_{v1} reveals the following differing sub-formula of the input partition for the path summarized above:

$$\begin{aligned}
IP_{debug}(L, S1) \wedge L1 == log_{debug}(L, S1) \wedge \\
IP_{isDebugEnabled}(L1) \wedge \\
RET_{isDebugEnabled}(L1) == FALSE \wedge \\
IP_{debug}(L1)
\end{aligned}$$

and the effects constraint is $LOG == log_{debug}(L1, S2)$, where L is the symbolic variable representing the initial value of `log`. These are isomorphic to the sub-formula that differ from Δ_{v0} . By

renaming CC and $CC1$ to L and $L1$ the difference is reduced to the pair of formula:

$$\begin{aligned} L1 &== \log_{debug}(L, S1) \wedge LOG == \log_{debug}(L1, S2) \\ CC1 &== catClass_{debug}(CC, S1) \wedge \\ CATCLASS &== catClass_{debug}(CC1, S2) \end{aligned}$$

which concisely characterizes the conditions under which the versions are equivalent. Specifically, if a pair of calls to `debug()` on a `log` with two strings yields the same cumulative effects as a pair of calls to `debug()` on a `catClass` with those same strings then the versions of `clear()` are equivalent.

We believe that strategies that isolate differences within a delta, e.g., by differencing sets of conjuncts, and then apply further techniques, e.g., identifying structural similarities and variable renamings, offer one approach to developing compact characterizations of version changes. In our study of JMeter version histories, this type of change occurred in 193 of the 572 methods modified between versions 0 and 1.

7. RELATED WORK

The problem of differencing programs has been widely studied. Tools that difference various representations of programs source code [2, 9, 17] are used everyday by practitioners to detect changed source files when using CM systems and to construct software patches. These tools also are used as the first-phase in more in-depth analyses of program differences and the impact that a change may have on other parts of the program. Using source difference as a starting point, multiple researchers have investigated approaches that identify an approximate characterization of the source code whose execution behavior may differ as a result of the change, e.g., [19, 20]. While useful, these techniques suffer from imprecision which can cause large parts of a program to be identified as potentially different. Our work focuses on developing more precise characterizations of program differences that offer the potential to reduce the cost of software maintenance activities.

In recent years, researchers, e.g., [15, 29], have begun to consider the challenges and opportunities that lie in targeting program analysis to version histories. Notkin identified several advantages of what he terms *longitudinal program analysis*. For example, program analysis cost can be amortized across multiple versions by reusing analysis results, analyses can target version differences, and analyses that might otherwise be intractable may become tractable when they focus on closely related program versions. We believe that DSE is the first instance of a longitudinal analysis framework that realizes all of these benefits.

DSE's approach to compare program versions was in part inspired by the work of Siegel et al. [22] who apply a combination of model checking and symbolic execution customized for floating point calculations, to verify the equivalence of sequential and parallel versions of a program solving a numerical computation. Their notion of equivalence is tunable, but it always captures the essential idea that the computed results are the same for the parallel and sequential programs for all inputs. In Siegel et al.'s work the exploration of concurrent process schedules is crucial to judging the equivalence of a sequential program run with the corresponding set of possible parallel program runs. While we implemented DSE in the JPF model checking framework to take advantage of its rich and flexible analysis infra-structure, our main objective was not to explore thread schedules. Instead, DSE provides a more comprehensive differential program analysis in that it characterizes the incompleteness of reasoning, proposes automated over-approximating abstract summaries as a means for addressing in-

completeness and scaling analysis, and focuses as much on delta characterization as on equivalence. In the context of reasoning about software versions, where many changes are intended to yield different program behavior, this focus on differences is essential.

In recent work, Siegel and Rossi apply the above technique to reason about the correctness of sequential and parallel versions of a non-trivial scientific computation [24]. This is achieved by manually constructing a model that captures the computation performed by the programs and that is amenable to analysis by their MPI-SPIN model checker [21]. As they detail in [23], in constructing the model they analyze the program for opportunities to reduce the cost of symbolic execution. One such reduction involves identifying common blocks of code in the sequential and parallel versions and modeling them as body-less functions which are uninterpreted. DSE automates the identification of common blocks between versions and the calculation of uninterpreted function signatures – both of these steps are integrated into the DSE methodology which requires no manual intervention.

In the context of test suite evolution, Apiwattanapong et al. [3] proposed a technique to generate test suite augmentation requirements with the assistance of symbolic evaluation. The technique identifies program statements in a dependency graph within a specified distance from a change, applies symbolic evaluation to capture the potential program states at the identified statements, and produces test requirements based on a pairwise comparison of the captured program states across the two program versions. In terms of scope, as exemplified in Section 6, DSE is more general in that it can be applied to a variety of clients including test suite augmentation. At a conceptual level, DSE operates on summaries rather than program states, which avoids the imprecision that comes with matching statements and corresponding states, enables the definition of equivalences, and helps to identify the conditions under which the analysis completely covers program behavior and when it fails to do so.

Symbolic equivalence checking is a well-established technique for comparing alternate hardware designs for functional equivalence, e.g., [4]. Researchers have adapted those ideas to reason about assembly code [6] using symbolic execution. While this work makes use of uninterpreted functions, they do not apply it to abstract common portions of code as is done in DSE. Furthermore, they do not consider the issues of delta characterization and incompleteness that are addressed in DSE.

More broadly, researchers have explored the general problem of semantic differencing. For example, Jackson and Ladd [12] build a symbolic characterization of program differences using an unsound analysis technique. Our abstract summaries allow soundness to be achieved when otherwise it could not and, in cases where summaries are incomplete, DSE characterizes the unsoundness through relative notions of equivalence and delta.

8. CONCLUSIONS

We have introduced differential symbolic execution as an approach for precisely calculating behavioral differences between versions of program methods. At the core of DSE are notions of method equivalence and deltas. We define equivalences and deltas that are sensitive only to black-box behavior and that consider a measure of white-box behavior. Variants of these equivalences accommodate the kinds of imprecision that commonly arise when performing symbolic execution of real Java software systems.

Building on the foundational DSE concepts, we present three classes of alternative formulations of DSE and describe tool support for two of those approaches. We illustrate the application of those tools to reason about selected changes from program version histo-

ries. For each such change, we describe a potential client analysis that leverages DSE results to support the evolution of test suites, to provide assurance that refactorings do not change program behavior, and to characterize the nature of a version change.

This work is the first step in focusing the recent significant advances in symbolic execution on problems of software evolution. Our future work will further develop DSE algorithms, engineer scalable implementations of those algorithms, develop automated support for DSE client applications, and then study the cost and effectiveness of DSE in automating software maintenance tasks.

9. REFERENCES

- [1] S. Anand, P. Godefroid, and N. Tillmann. Demand-driven compositional symbolic execution. In *Proceedings of the International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, 2008.
- [2] T. Apiwattanapong, A. Orso, and M. J. Harrold. Jdiff: A differencing technique and tool for object-oriented programs. *Automated Software Engineering*, 14(1):3–36, 2007.
- [3] T. Apiwattanapong, R. Santelices, P. K. Chittimalli, A. Orso, and M. J. Harrold. Matrix: Maintenance-oriented testing requirements identifier and examiner. In *Proceedings of the Testing: Academic and Industrial Conference – Practice and Research Techniques*, pages 137–146, 2006.
- [4] R. E. Bryant, S. German, and M. N. Velev. Processor verification using efficient reductions of the logic of uninterpreted functions to propositional logic. *ACM Transactions Computational Logic*, 2(1):93–134, 2001.
- [5] Choco. Main-page Choco. <http://choco-solver.net>, 2008.
- [6] D. Currie, X. Feng, M. Fujita, A. J. Hu, M. Kwan, and S. Rajan. Embedded software verification using symbolic execution and uninterpreted functions. *International Journal of Parallel Programming*, 34(1):61–91, 2006.
- [7] CVC3. CVC3 page. <http://www.cs.nyu.edu/acsys/cvc3>, 2008.
- [8] B. Daniel, D. Dig, K. Garcia, and D. Marinov. Automated testing of refactoring engines. In *Proceedings of the the 6th joint meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering*, pages 185–194, 2007.
- [9] Diffutils. Comparing and merging files. <http://www.gnu.org/software/diffutils>, 2002.
- [10] M. Fowler. Refactoring home page. <http://refactoring.org>, 2007.
- [11] P. Godefroid. Compositional dynamic test generation. In *Proceedings of the ACM Symposium on Principles of Programming Languages*, pages 47–54, 2007.
- [12] D. Jackson and D. A. Ladd. Semantic Diff: A tool for summarizing the effects of modifications. In *Proceedings of the International Conference on Software Maintenance*, pages 243–252, 1994.
- [13] JMeter. Jmeter–Apache JMeter. <http://jakarta.apache.org/jmeter/>, 2008.
- [14] J. C. King. Symbolic execution and program testing. *Communications of the ACM*, 19(7):385–394, 1976.
- [15] D. Notkin. Longitudinal program analysis. In *Proceedings of the ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering*, 2002.
- [16] C. S. Păsăreanu, P. C. Mehltz, D. H. Bushnell, K. Gundy-Burlet, M. Lowry, S. Person, and M. Pape. Combining unit-level symbolic execution and system-level concrete execution for testing NASA software. In *Proceedings of the International Symposium on Software Testing and Analysis*, pages 15–25, 2008.
- [17] S. Raghavan, R. Rohana, D. Leon, A. Podgurski, and V. Augustine. Dex: a semantic-graph differencing tool for studying changes in large code bases. In *Proceedings of the International Software Maintenance Conference*, pages 188–197, 2004.
- [18] V. P. Ranganath and J. Hatcliff. Slicing concurrent Java programs using Indus and Kaveri. *Software Tools for Technology Transfer*, 9(5-6):489–504, 2007.
- [19] X. Ren, B. G. Ryder, M. Stoerzer, and F. Tip. Chianti: a change impact analysis tool for Java programs. In *Proceedings of the 27th International Conference on Software Engineering*, pages 664–665, 2005.
- [20] G. Rothmel and M. J. Harrold. A safe, efficient regression test selection technique. *ACM Transactions on Software Engineering and Methodology*, 6(2):173–210, 1997.
- [21] S. F. Siegel. MPIN-SPIN page. <http://vsl.cis.udel.edu/mpi-spin/>, 2008.
- [22] S. F. Siegel, A. Mironova, G. S. Avrunin, and L. A. Clarke. Using model checking with symbolic execution to verify parallel numerical programs. In *Proceedings of the 2006 International Symposium on Software Testing and Analysis*, pages 157–168, 2006.
- [23] S. F. Siegel, S. E. Moelius, and L. F. Rossi. A case study in the use of model checking to verify parallel scientific software. Technical Report UDEL-CIS 2007/343, Department of Computer and Information Sciences, University of Delaware, Nov. 2007.
- [24] S. F. Siegel and L. F. Rossi. Analyzing BlobFlow: A case study using model checking to verify parallel scientific software. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface, 15th European PVM/MPI User’s Group Meeting, Proceedings*, volume 5205 of LNCS. Springer, 2008. To appear.
- [25] Siena. Siena: A wide-area event notification service. <http://serl.cs.colorado.edu/~serl/dot/siena.html>, 2008.
- [26] SIR. Software-artifact infrastructure repository: Home. <http://sir.unl.edu>, 2008.
- [27] W. Visser, C. S. Păsăreanu, and S. Khurshid. Test input generation with Java PathFinder. In *Proceedings of the International Symposium on Software Testing and Analysis*, pages 97–107, 2004.
- [28] W. Visser, C. S. Păsăreanu, and R. Pelánek. Test input generation for Java containers using state matching. In *Proceedings of the International Symposium on Software Testing and Analysis*, pages 37–48, 2006.
- [29] J. Winstead and D. Evans. Towards differential program analysis. In *Proceedings of the ICSE Workshop on Dynamic Analysis*, 2003.